Investigation of Variance Reduction in Ensemble Methods

Johan Beatty

1 Introduction

In supervised machine learning problems we are trying to approximate some unknown function f^* with an estimated model $\hat{f}$. The bias of a trained learner $\hat{f}$ is a measure of how far the average prediction is from the true value of the function f^*. The variance of an trained learner is a measure of how much $\hat{f}$ changes when trained on different training set.

$$E[(f^* - \hat{f})^2] = \text{Bias}^2 + \text{Var} + \text{Var}(\epsilon) \tag{1}$$

The expected test MSE can be decomposed into 3 components as can be seen from equation (1). The Var, Bias^2 and the irreducible error Var(ϵ). Here $E[(f^* - \hat{f})^2]$ is a theoretical quantity that would be obtained from estimating f^* from many training samples. Both bias and variance affect the predictive performance of a trained learner, and we want to minimize these. However, this leads us to a fundamental challenge, that it is generally not possible to reduce both at the same time. This is what is known as the bias variance trade-off (James et al., 2013) [4].

Bootstrap is a technique that can be applied to estimate uncertainty around statistical estimates when underlying distribution is unknown (Efron, 1979) [3]. The idea is to create many different samples from a set of training data by sampling with replacement, these new sample are called bootstrap samples. Then you estimate the statistic for each bootstrap sample and create confidence intervals to quantify the uncertainty around the estimate. This idea was then expanded on by fitting models to each bootstrap sample, these models are known as simple or "weak" learners. This creates an ensemble of simple learners, the idea is to average the predictions from ensemble to get a final prediction for the bootstrapped models. This is what is known as bootstrap aggregation or "bagging" (Breiman, 1996) [1]. The idea behind bagging is to reduce the variance of a model by averaging many predictions. The theoretical motivation comes from the fact that we know from that averaging n-independent observations shrinks the variance of any estimate.

If $X_1, \ldots, X_n$ are independent observations with variance σ^2 :

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$$

$$\text{then } \text{Var}(\bar{X}) = \frac{\sigma^2}{n} \to 0 \text{ as } n \to \infty \tag{2}$$

So in theory bagging should decrease the variance while keeping the bias constant. This study aims to investigate this claim by comparing the bias and variance of simple learners to ensembles of simple learners using real world data. Bootstrap will be used in order to quantify the bias and variance of the learners.

In order to investigate this claim we will be comparing how three different base learners behave in ensembles:

- K-nearest neighbor (KNN)

- Decision tree
- Logistic Regression

These models were chosen for a few reasons. The first one being computational cost. One of the drawbacks of bagging is obviously the computational cost. These models are simple and should fit fairly quick under normal conditions. Secondly they don't introduce any additional source of variance reduction when paired with bagging, such as a random forest or boosting methods could. This ensures that we can account a reduction in variance solely due to the ensembling process. Lastly, comparing these models will allow us to investigate ensembling on models with different variance profiles. Decision trees are known to have high variance while logistic regression is know to have low variance and KNN sits somewhere in between.

The data used in this experiment is the breast cancer data from Scikit-learn [5]. The models will be tasked with a binary classification problem of predicting malignant tumors from benign. This data was chosen primarily for it's quality and ease of use. No manipulation or pre-processing is needed for the purpose of this study.

The findings from the experiment shows that ensembling does seem to lower the variance compared to simple learners, although to varying degrees depending on the simple learner. Decision trees saw the largest reduction in variance while the KNN model also saw a fairly large reduction in variance compared to the base learner. Logistic regression saw the least amount of reduction in variance compared to the base learner. The bias did seem to stay somewhat unaffected by the ensembling process.

2 Background

In the introduction we introduced the bias-variance decomposition in terms of the mean squared error. MSE is only defined in the regression setting. For a binary classification problem, model fit is evaluated by the test error rate:

$$\text{Ave}(I(y \neq \hat{y}))$$

Also known as 0-1 loss. The bias and variance for 0-1 loss can be defined as (Dietterich and Kong, 1995) [2]:

$$\text{Bias} = \begin{cases} 1 & \text{if } y \neq E[\hat{y}] \\ 0 & \text{otherwise} \end{cases}$$

$$\text{Variance} = P(\hat{y} \neq E[\hat{y}])$$

To quantify the bias and variance we will use bootstrap. We generate B bootstrap samples and obtain predictions $\hat{y}^{(1)}, \ldots, \hat{y}^{(B)}$ for each test observation.

$$\widehat{\text{Bias}} = \frac{1}{m} \sum_{i=1}^{m} I(y_i \neq E[\hat{y}_i]) \tag{3}$$

$$\widehat{\text{Variance}} = \frac{1}{m} \sum_{i=1}^{m} \left[\frac{1}{B} \sum_{b=1}^{B} I(\hat{y}_i^{(b)} \neq E[\hat{y}_i]) \right] \tag{4}$$

where m is the number of test observations and $E[\hat{y}_i]$ denotes the "main" prediction of an estimated model of test point i.

2.1 K-Nearest Neighbors

The k-nearest neighbor (KNN) is a simple non-parametric algorithm. To classify a new observation x:

1. Find the k nearest points in the training data using the L2 norm (Euclidean distance)
2. Look at the classes of the k nearest points
3. Predict observation x to the most common class among the k nearest points

2.2 Decision Tree

The decision tree classifier use a binary workflow. It starts by looking at the full data and decides what variable to split on by evaluating the Gini Index

$$\sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}) \tag{5}$$

where $\hat{p}_{mk}$ represents the proportion of training observations in region m that belong to class k. From looking at equation (5). we can see that the Gini index will take on small values for large and small values of $\hat{p}_{mk}$. Gini index is also know as a measure of node purity. Once a variable is chosen the data will be split such that the Gini index is minimized. The algorithm will keep splitting until some stopping criterion is met, or until fully grown. I.e. 1 observation in each leaf node. Usually a decision tree in fully grown and then pruned back. However in this study no tree pruning was use, instead a fixed value for the parameter maximum tree depth was predetermined using cross validation.

2.3 Logistic Regression

Logistic regression is a parametric method that belongs to the GLM family. It models the conditional probability of an observation belonging to a class given a set of predictors. It is a generalization of linear regression where the predictors are mapped to the binary outcome variable through a link function, often a sigmoid function (James et al., 2013) [4].

3 Methodology

3.1 Data

The data contains a total of 569 observations and has 30 features describing the size and physical appearance of tumors. The target variable describes if the tumor is malignant or benign.

3.2 Model fitting

The analysis was conducted using python. First of, the data was randomly split into 60% training data and 40% testing data. The reason for this rather beefy split is that initially on a 80/20 split, the models simply were doing too good of a job, leaving little room for evaluating the error. Therefor a decision was made to make this split larger in order to increase the difficulty of the task.

The parameters for the KNN and decision tree models was determined using 5-fold cross validation on the training data. The number of k-neighbors considered was varied from 1 to 20, and the maximum tree depth was varied from 1 to max depth (fully grown tree). From table 1 we can

see the results from the cross validation of the decision tree. In a traditional predictive modeling setting either 4 or 5 looks like the best option for tree depth. However in this case intentionally "suboptimal" parameters were chosen to ensure some excess variance in the models. Therefore a tree depth of 8 is chosen for the decision tree. In table 2 we can see the results from the KNN cross validation. It seems like 3 is probably the best choice for the predictive setting. Therefor 2 is chosen for the experiment to increase the variance.

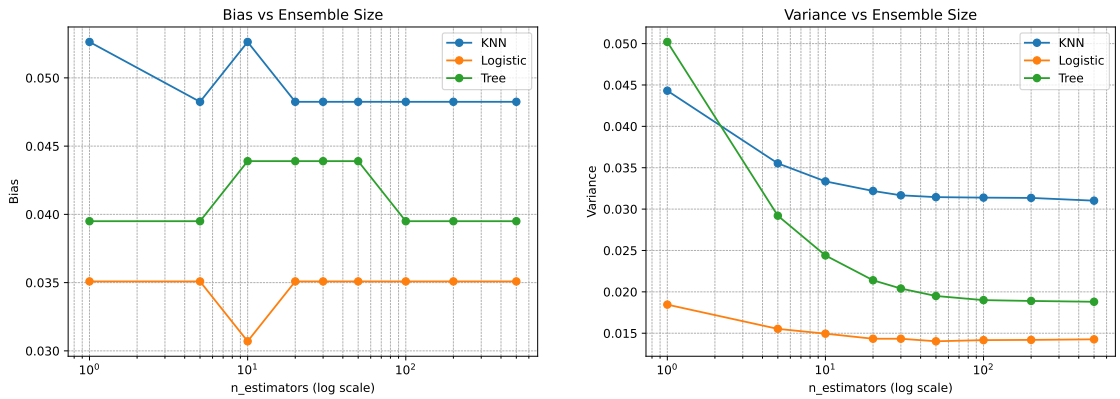
Once hyper-parameters have been determined, the models were fit and estimates of bias and variance is obtained. A function for each model was created. The workflow of the functions is as follows:

1. Takes training data, test data, number of bootstrap samples B, and hyperparameter values as inputs
2. Samples with replacement from the training data to create a bootstrap sample
3. Fits a model to the bootstrapped sample with fixed hyperparameter
4. Makes predictions on the test data with the fitted model
5. Repeats steps 2-4 B times, creating an $m \times B$ array where each column contains predictions from one fitted model (m is the number of test observations)
6. Makes final prediction by majority voting: the mode of each row is the final prediction, yielding a vector of length m with the final predictions.
7. Estimates bias and variance from the $m \times B$ array by equation (3) and (4)

The number of learners (number of estimators) in the ensembles are varied in increasing increments from 5 to 500. 500 bootstrap samples is used each run to estimate the bias and variance. This means that the last run estimates 500x500 = 250 000 simple learners. The run time was roughly 22 minutes for the decision trees, 14 minutes for KNN and 9 minutes for the logistic regression learners.

4 Results

In Figure 1 (b) we can see the estimated biases from each model plotted against the log of the ensemble sizes. From the plots we can see that the estimated biases seem to stay fairly constant. The estimated biases seem to jump between two distinct values for each model. Furthermore, the distance between these two values is 0.0044, the same for each model. If we look closer at how bias is calculated in equation (3) and consider the size of the test set which is 228 observations. We can see that $1/228 \approx 0.0044$, i.e. the change in bias is due to 1 classification flipping. So even tho the estimated biases appear to move around a bit it is safe to say that they seem to stay constant. Tables of the results can be found in the appendix.



(a) Bias

(b) Variance

Figure 1: Bias and variance vs ensemble size (log scale) for KNN, logistic regression, and decision trees.

The estimated variance of each model is plotted in Figure 1(b) against ensemble sizes. From the plot we can see that each model seem to decrease in variance as the ensemble sizes increase. By inspecting the lines it is clear that the decision tree ensembles see the fastest and largest reduction in variance as the ensemble sizes increase. The KNN ensembles also see a fairly noticeable reduction in variance. Logistic regression ensembles see a very small reduction in variance from the base learner to an ensemble of size 5 and then barley noticeable changes in variance after that.

5 Discussion

In my mind the base learner variance and bias should be estimated by simply bootstrapping the original training data and fitting a single model to each bootstrap sample. However i believe comparing the variance of this base learner to the ensembles might be problematic. The variance of the ensembles is estimated by bootstrapping. We know form theory that the number of unique data points in a bootstrapped sample is reduced by roughly 30%. So if an ensemble of learners is fit to an already bootstrapped sample. That means that each model in the ensembles is fit to a "double" bootstrapped sample. This artificially "shrinks" the sample size by further reducing the number of unique data points available. From equation (2) we get that this should increase the variance of our estimate. So the issue then becomes if we compare to a true base learner. That the base learner will estimate it's variance by bootstrapping the full training data while the ensemble variances will be estimated by bootstrapping an already bootstrapped sample. My thinking is that this yields an unfair and biased comparison. That is why in this experiment the bias and variance of the base learners has been estimated by setting n_estimator in the function equal to 1. I.e. the bias and variance is estimated by double bootstrap even for the base learner. I believe it creates a more fair setting for this experiment. There is no perfect way when to estimate bias and variance on real life data. With simulated data we could make a more fair and correct comparison.

Now if we assume that our approach is correct then the results do, to some degree confirm the claim that ensembling reduces variance whilst keeping bias constant. Although to varying degrees of effectiveness. Just like expected, the high variance decision trees benefited greatly from ensembling. The logistic regression ensembles seemed to be rather ineffective with only minor reduction in variance. The effectiveness of the KNN ensembles lie somewhere in between. The bias seemed to stay constant for all models. One thing to keep in mind is that these results have

been produced under conditions for ensembling to "succeed". In a normal setting we probably would not see the results we see for the decision trees for example. Since hyperparameter would have been tuned optimally. Another note is that a smaller dataset was chosen intentionally to give more power to the ensembles. The idea is that the more observations we have, the less effective ensembling becomes. For future research i would suggest doing this with simulated data instead. Perhaps trying other models as well. It would also be interesting to see how much sample size affects ensembling. So fo future research one could vary the sample size and investigate ensembling.

A.I disclaimer. Chat GPT has been assisting in the creation of the bias/variance estimation functions as well as plotting. It has been used for citations, creations of tables and latex notation for the report.

References

[1] Leo Breiman. Bagging predictors. *Machine Learning*, 24, 1996.

[2] Thomas G. Dietterich and Eun Bae Kong. Machine learning bias, statistical bias, and statistical variance of decision tree algorithms. Technical report, Oregon State University, Department of Computer Science, 1995.

[3] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1), 1979.

[4] Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. *An Introduction to Statistical Learning: with Applications in Python*. Springer, New York, 2023.

[5] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

A results

Table 1: Bias and variance estimates for ensembles of decision trees, max_depth=5)

n_estimators	**Bias**	**Variance**
1	0.0395	0.0502
5	0.0395	0.0292
10	0.0439	0.0244
20	0.0439	0.0214
30	0.0439	0.0204
50	0.0439	0.0195
100	0.0395	0.0190
200	0.0395	0.0189
500	0.0395	0.0188

Table 2: Bias and variance estimates for KNN ensembles with varying number of estimators ($k = 2$).

n_estimators	**Bias**	**Variance**
1	0.052632	0.044307
5	0.048246	0.035526
10	0.052632	0.033351
20	0.048246	0.032193
30	0.048246	0.031667
50	0.048246	0.031447
100	0.048246	0.031386
200	0.048246	0.031351
500	0.048246	0.031018

Table 3: Bias and variance estimates for logistic regression ensembles.

n_estimators	Bias	Variance
1	0.035088	0.018456
5	0.035088	0.015526
10	0.030702	0.014947
20	0.035088	0.014333
30	0.035088	0.014333
50	0.035088	0.014026
100	0.035088	0.014167
200	0.035088	0.014193
500	0.035088	0.014263

B Code

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline
from sklearn.tree import DecisionTreeClassifier
import pandas as pd
from ISLP import load_data
import numpy as np
from sklearn.neighbors import KNeighborsClassifier as KNN
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import (
    cross_validate,
    KFold,
    ShuffleSplit
)

data = load_breast_cancer()
data = pd.DataFrame(data.data, columns=data.feature_names)

data.head()

# Create table of malignant vs benign counts
data['target'] = load_breast_cancer().target
data['target'].value_counts()

train, test = train_test_split(data,
                               test_size = 0.4,
                               random_state = 100,
                               stratify = data['target'])

x_train = train.drop(columns=['target'])
y_train = train['target']
x_test = test.drop(columns=['target'])
y_test = test['target']
```

```
x_train = np.asarray(x_train)
y_train = np.asarray(y_train).astype(int)
x_test  = np.asarray(x_test)
y_test  = np.asarray(y_test).astype(int)

n_train = x_train.shape[0]
n_test  = x_test.shape[0]

##########################################################################
# CROSS_VALIDATION
##########################################################################

cv = KFold(
    n_splits = 5,
    shuffle = True,
    random_state = 100   # for reproducible folds
)

max_depth_values = [1, 2, 3, 4, 5, 6, 8, 10, None]

results_dt = []

for depth in max_depth_values:

    dt = DecisionTreeClassifier(
        max_depth = depth,
        random_state = 100
    )

    cv_res = cross_validate(
        dt,
        x_train,
        y_train,
        cv = cv,
        scoring='accuracy',
        return_train_score=True,
        n_jobs=-1,
    )

    results_dt.append({
        "max_depth": depth,
        "mean_train_acc": cv_res["train_score"].mean(),
        "mean_test_acc":  cv_res["test_score"].mean(),
    })

results_dt = pd.DataFrame(results_dt)
print(results_dt)

k = [1, 2, 3, 4, 5, 6, 8, 10, 15, 20]
```

```
results_knn = []

for neighbors in k:
    knn = KNN(n_neighbors = neighbors)

    cv_res = cross_validate(
        knn,
        x_train,
        y_train,
        cv = cv,
        scoring='accuracy',
        return_train_score=True,
        n_jobs=-1,
    )
    results_knn.append({
        "n_neighbors": neighbors,
        "mean_train_acc": cv_res["train_score"].mean(),
        "mean_test_acc":  cv_res["test_score"].mean(),
    })

results_knn = pd.DataFrame(results_knn)
print(results_knn)

##########################################################################
# KNN BAGGING
##########################################################################

def knn_ensemble(x_train, y_train, x_test, n_estimators=50, k=2, seed=100):
    rng = np.random.default_rng(seed)

    base_preds = np.zeros((n_test, n_estimators), dtype=int)

    for m in range(n_estimators):
        idx = rng.integers(0, n_train, size=n_train)
        X_boot = x_train[idx]
        y_boot = y_train[idx]

        model = make_pipeline(
            StandardScaler(),
            KNN(n_neighbors=k)
        )
        model.fit(X_boot, y_boot)

        base_preds[:, m] = model.predict(x_test)

    ensemble_pred = (base_preds.mean(axis=1) >= 0.5).astype(int)
    return ensemble_pred, base_preds

##########################################################################
```

```
# TREE BAGGING
##############################################################################

def tree_ensemble(x_train, y_train, x_test, n_estimators=50, max_depth=None, seed=100):
    rng = np.random.default_rng(seed)

    base_preds = np.zeros((n_test, n_estimators), dtype=int)

    for m in range(n_estimators):
        # Bootstrap sample within the training set
        idx = rng.integers(0, n_train, size=n_train)
        X_boot = x_train[idx]
        y_boot = y_train[idx]

        # Fit one tree on this bootstrap sample
        tree = DecisionTreeClassifier(
            max_depth=max_depth,
            random_state=seed + m  # make tree construction deterministic
        )
        tree.fit(X_boot, y_boot)

        base_preds[:, m] = tree.predict(x_test)

    # Majority vote across trees
    ensemble_pred = (base_preds.mean(axis=1) >= 0.5).astype(int)
    return ensemble_pred, base_preds
##############################################################################
# LOGISTIC REGRESSION BAGGING
##############################################################################

def logistic_ensemble(x_train, y_train, x_test, n_estimators=50, seed=100):
    rng = np.random.default_rng(seed)

    n_train = x_train.shape[0]
    n_test  = x_test.shape[0]

    base_preds = np.zeros((n_test, n_estimators), dtype=int)

    for m in range(n_estimators):
        idx = rng.integers(0, n_train, size=n_train)
        X_boot = x_train[idx]
        y_boot = y_train[idx]

        model = make_pipeline(
            StandardScaler(),
            LogisticRegression(random_state=seed + m, max_iter=5000)
        )

        model.fit(X_boot, y_boot)
        base_preds[:, m] = model.predict(x_test)
```

```
    ensemble_pred = (base_preds.mean(axis=1) >= 0.5).astype(int)
    return ensemble_pred, base_preds

##########################################################################
# BOOTSTRAP ESTIMATEION FOR BAGGED TREES
##########################################################################

def boot_tree_bagged(x_train, y_train, x_test, y_test,
                     n_boot=200, seed=100, max_depth=None, n_estimators=50):
    rng_outer = np.random.default_rng(seed)

    # Each column = one bagged-tree ensemble trained on an outer bootstrap sample
    preds = np.zeros((n_test, n_boot), dtype=int)

    for b in range(n_boot):
        # Outer bootstrap sample of the training set
        idx_outer = rng_outer.integers(0, n_train, size=n_train)
        X_outer = x_train[idx_outer]
        y_outer = y_train[idx_outer]

        # Inner bagging: train an ensemble on X_outer/y_outer
        ensemble_pred, _ = tree_ensemble(
            X_outer, y_outer, x_test,
            n_estimators=n_estimators,
            max_depth=max_depth,
            seed=seed + b
        )

        preds[:, b] = ensemble_pred

    # 0-1 loss decomposition (main prediction = majority vote over outer rounds)
    means = preds.mean(axis=1)
    main = (means >= 0.5).astype(int)

    bias_vec = np.where(main != y_test, 1, 0)
    disagreement = np.where(preds != main[:, None], 1, 0)
    var_vec = disagreement.mean(axis=1)

    return bias_vec.mean(), var_vec.mean(), preds

bias_bag_tree, var_bag_tree , preds_bag_knn = boot_tree_bagged(
    x_train, y_train, x_test, y_test,
    n_boot=500,
    seed=100,
    max_depth=8,
    n_estimators=5
)

print(bias_bag_tree, var_bag_tree)
```

```
n_trees = [1, 5, 10, 20, 30, 50, 100, 200, 500]
indexx = [0, 1, 2, 3, 4, 5, 6, 7, 8]

trees = np.zeros((len(n_trees), 3))

for i in indexx:
    bias_bag_tree, var_bag_tree , preds_bag_knn = boot_tree_bagged(
    x_train, y_train, x_test, y_test,
    n_boot=500,
    seed=100,
    max_depth=5,
    n_estimators=n_trees[i]

)
    trees[i, 0] = n_trees[i]
    trees[i, 1] = bias_bag_tree
    trees[i, 2] = var_bag_tree

trees = pd.DataFrame(trees)
print(trees)

##############################################################################
# BOOTSTRAP ESTIMATION FOR BAGGED KNN
##############################################################################

def boot_knn_bagged(x_train, y_train, x_test, y_test,
                    n_boot=200, seed=100, k=2, n_estimators=50):

    rng_outer = np.random.default_rng(seed)

    # Each column = one *bagged-KNN* model trained in one bootstrap round
    preds = np.zeros((n_test, n_boot), dtype=int)

    for b in range(n_boot):
        # Outer bootstrap = resample the training set
        idx_outer = rng_outer.integers(0, n_train, size=n_train)
        X_outer = x_train[idx_outer]
        y_outer = y_train[idx_outer]

        # Inner bagging = fit an ensemble on that resampled training set
        # Use a different seed per round so the inner bagging differs each time
        ensemble_pred, _ = knn_ensemble(
            X_outer, y_outer, x_test,
            n_estimators=n_estimators,
            k=k,
            seed=seed + b
        )

        preds[:, b] = ensemble_pred
```

```
    # 0-1 loss decomposition (mode/majority-vote main prediction)
    means = preds.mean(axis=1)
    main = (means >= 0.5).astype(int)

    bias_vec = np.where(main != y_test, 1, 0)
    disagreement = np.where(preds != main[:, None], 1, 0)
    var_vec = disagreement.mean(axis=1)

    return bias_vec.mean(), var_vec.mean(), preds

neighbours = np.zeros((len(n_ensemb), 3))

for i in indexx:
    bias_bag_knn, var_bag_knn, preds_bag_knn = boot_knn_bagged(
        x_train, y_train, x_test, y_test,
        n_boot=500,
        seed=100,
        k=2,
        n_estimators=n_ensemb[i]
    )
    neighbours[i, 0] = n_ensemb[i]
    neighbours[i, 1] = bias_bag_knn
    neighbours[i, 2] = var_bag_knn

neighbours = pd.DataFrame(neighbours)
print(neighbours)

##############################################################################
# BOOTSTRAP ESTIMATION FOR BAGGED LOGISTIC REGRESSION
##############################################################################

def boot_logistic_bagged(x_train, y_train, x_test, y_test,
                         n_boot=200, seed=100, n_estimators=50):

    rng_outer = np.random.default_rng(seed)

    # Each column = one bagged-logistic model trained in one bootstrap round
    preds = np.zeros((n_test, n_boot), dtype=int)

    for b in range(n_boot):
        # Outer bootstrap = resample the training set
        idx_outer = rng_outer.integers(0, n_train, size=n_train)
        X_outer = x_train[idx_outer]
        y_outer = y_train[idx_outer]

        # Inner bagging = fit an ensemble on that resampled training set
        # Use a different seed per round so the inner bagging differs each time
        ensemble_pred, _ = logistic_ensemble(
            X_outer, y_outer, x_test,
```

```
            n_estimators=n_estimators,
            seed=seed + b
        )

        preds[:, b] = ensemble_pred

    # 0-1 loss decomposition (mode/majority-vote main prediction)
    means = preds.mean(axis=1)
    main = (means >= 0.5).astype(int)

    bias_vec = np.where(main != y_test, 1, 0)
    disagreement = np.where(preds != main[:, None], 1, 0)
    var_vec = disagreement.mean(axis=1)

    return bias_vec.mean(), var_vec.mean(), preds

log_reg = np.zeros((len(n_ensemb), 3))

for i in indexx:
    bias_bag_log, var_bag_log, preds_bag_log = boot_logistic_bagged(
        x_train, y_train, x_test, y_test,
        n_boot=500,
        seed=100,
        n_estimators=n_ensemb[i]
    )
    log_reg[i, 0] = n_ensemb[i]
    log_reg[i, 1] = bias_bag_log
    log_reg[i, 2] = var_bag_log

log_reg = pd.DataFrame(log_reg)
print(log_reg)


##########################################################################
# PLOTTING
##########################################################################

knn_data = [
    (1,   0.052632, 0.044307),
    (5,   0.048246, 0.035526),
    (10,  0.052632, 0.033351),
    (20,  0.048246, 0.032193),
    (30,  0.048246, 0.031667),
    (50,  0.048246, 0.031447),
    (100, 0.048246, 0.031386),
    (200, 0.048246, 0.031351),
    (500, 0.048246, 0.031018),
]

logit_data = [
```

```
    (1,   0.035088, 0.018456),
    (5,   0.035088, 0.015526),
    (10,  0.030702, 0.014947),
    (20,  0.035088, 0.014333),
    (30,  0.035088, 0.014333),
    (50,  0.035088, 0.014026),
    (100, 0.035088, 0.014167),
    (200, 0.035088, 0.014193),
    (500, 0.035088, 0.014263),
]

tree_data = [
    (1,   0.0395, 0.0502),
    (5,   0.0395, 0.0292),
    (10,  0.0439, 0.0244),
    (20,  0.0439, 0.0214),
    (30,  0.0439, 0.0204),
    (50,  0.0439, 0.0195),
    (100, 0.0395, 0.0190),
    (200, 0.0395, 0.0189),
    (500, 0.0395, 0.0188),
]

knn_df = pd.DataFrame(knn_data, columns=['n_estimators', 'bias', 'variance'])
logit_df = pd.DataFrame(logit_data, columns=['n_estimators', 'bias', 'variance'])
tree_df = pd.DataFrame(tree_data, columns=['n_estimators', 'bias', 'variance'])

# Combine into one long dataframe (best for plotting)
knn_plot   = knn_df.assign(model="KNN")
logit_plot = logit_df.assign(model="Logistic")
tree_plot  = tree_df.assign(model="Tree")
df_all = pd.concat([knn_plot, logit_plot, tree_plot], ignore_index=True)

# ---- Plot 2: Variance ----
plt.figure()
for name, g in df_all.groupby("model"):
    plt.plot(g["n_estimators"], g["variance"], marker="o", label=name)

plt.xscale("log")
plt.xlabel("n_estimators (log scale)")
plt.ylabel("Variance")
plt.title("Variance vs Ensemble Size")
plt.legend()
plt.grid(True, which="both", linestyle="--", linewidth=0.5)

plt.tight_layout()
plt.savefig("variance_vs_ensemble.pdf")        # vector (great for LaTeX)
plt.savefig("variance_vs_ensemble.png", dpi=300)
plt.show()
```

```
# ---- Plot 1: Bias ----
plt.figure()
for name, g in df_all.groupby("model"):
    plt.plot(g["n_estimators"], g["bias"], marker="o", label=name)

plt.xscale("log")
plt.xlabel("n_estimators (log scale)")
plt.ylabel("Bias")
plt.title("Bias vs Ensemble Size")
plt.legend()
plt.grid(True, which="both", linestyle="--", linewidth=0.5)

plt.tight_layout()
plt.savefig("bias_vs_ensemble.pdf")
plt.savefig("bias_vs_ensemble.png", dpi=300)
plt.show()
##############################################################################
```